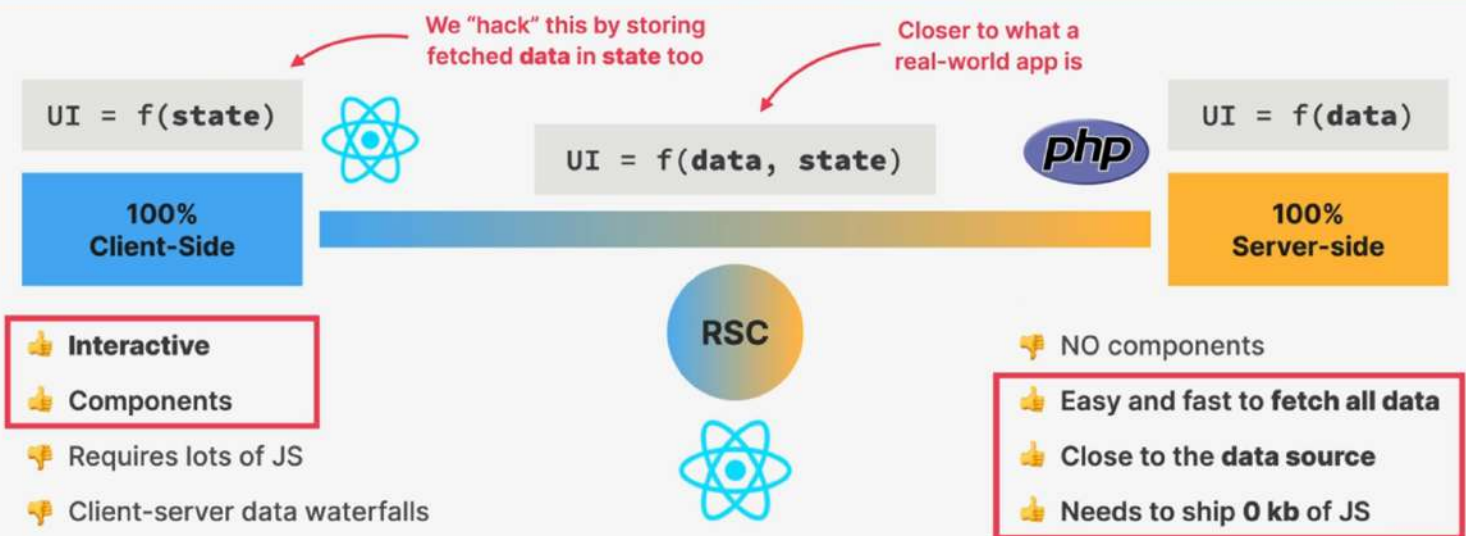


WHY REACT SERVER COMPONENTS?



🤔 What if we could take the best of both worlds?

🧩 The answer is a completely new React paradigm: **React Server Components (RSC)**

WHAT ARE REACT SERVER COMPONENTS?

REACT SERVER COMPONENTS (RSC)

- 👉 A new **full-stack architecture** for React apps
- 👉 Introduces the server as an integral part of React component trees: **server components**
- 👉 We write **frontend code** next to **backend code** in a natural way that “feels” like regular React
- 👉 RSC is **NOT** active by default in new React apps (e.g. Vite apps): it needs to be **implemented** by a framework like **Next.js** (“app router”)

Name of the
new **PARADIGM**

Name of the new
COMPONENT TYPE

1

CLIENT COMPONENTS

UI = f(**state**)

- 👉 “Regular” components
- 👉 Created with “**use client**” directive at the top of the module

2

SERVER COMPONENTS

UI = f(**data**)

- 👉 Components that are **only rendered on the server**
- 👉 Don’t make it into the bundle (**0 kb**)
- 👉 We can build the **back-end with React!**
- 👉 **Default** in apps that use the RSC architecture (like Next.js)

SERVER COMPONENTS VS. CLIENT COMPONENTS

	CLIENT COMPONENTS	“use client”	SERVER COMPONENTS	Default
⚡ State/hooks	✅ YES		❌ NO	
⬆️ Lifting state up	✅ YES		❌ N.A.	
⬇️ Props	✅ YES		✅ YES (Must be serializable when passed to client components. No functions or classes)	
📡 Data fetching	👉 Also possible, preferably with library		✅ Preferred. Use <code>async/await</code> in component	
📦 Can import	Only client components (can't go back in the client-server boundary)		Client and server components	
🖌️ Can render	Client components and server components <i>passed as props</i>		Client and server components	
🔄 When re-render?	On state change		On URL change (navigation)	

STATIC VS. DYNAMIC RENDERING

STATIC RENDERING

- 👉 HTML is generated at **built time**, or **periodically in the background** by re-fetching data (ISR, *more on this later* 🤖)
- 👉 Useful when data **doesn't change often** and is **not personalized** to user (e.g. product page)
- 👉 **Default rendering strategy** in Next.js (even when a page or component fetches data)
- 👉 When deployed to **Vercel**, each static route is automatically hosted on a CDN
- 👉 If all routes of an app are static, the entire app can be **exported as a static site** (SSG)

Content Delivery Network

DYNAMIC RENDERING

- 👉 HTML is generated at **request time** (for each new request reaches the server)
- 👉 Makes sense if:
 - 1 The **data changes frequently** and is **personalized** to the user (e.g. cart)
 - 2 Rendering a route requires information that **depends on request** (e.g. search params)
- 👉 A route **automatically switches to dynamic** rendering in certain conditions (*next slide* 🤖)
- 👉 When deployed to **Vercel**, each dynamic route becomes a serverless function

WHEN NEXT.JS SWITCHES TO DYNAMIC RENDERING

- ➡ Usually, developers **don't directly choose** whether a route should be static or dynamic. Next.js will **automatically switch to dynamic** rendering in the following scenarios:

- 1 The route has a **dynamic segment** (page uses **params**)
- 2 **searchParams** are used in the *page component* `/product?quantity=23`
- 3 **headers()** or **cookies()** are used in any of the route's *server components*
- 4 An **uncached data request** is made in any of the route's *server components*

- ➡ This is necessary because any of these values **can not be known** by Next.js at built time

- ➡ We can also **force** Next.js to render a route dynamically:

- ➡ `export const dynamic = 'force-dynamic';` from *page.js*
- ➡ `export const revalidate = 0;` from *page.js*
- ➡ `{ cache: 'no-store' }` added to a **fetch** request in any of the route's *server components*
- ➡ `noStore()` in any of the route's *server components*

These influence caching 4
More on this later ➡